04/04

Stack Datastructure

Stack is a linear datastructure , which follows last in first out (lifo)
Stack has only one end i.e top
Operations of Stack
push ---> inserting the data into stack is called push
pop ---> deleting of data from the stack is called Pop
peek ---> gives data pointed by top

Note : we always insert or delete the data from top

Push , pop ,peek takes timecomplexity of O(1)

Applications of Stack
1. Functions and returns
2. Reverse Printing of String using recursion
3. Checking for balanced parenthesis for a given expression
4. converting infix expression to postfix expression
5. converting infix expression to prefix expression
6. Evaluting postfix expression
7. Evaluting prefix expression

How do we implement stack data structure
1. using arrays
2. using link list

When stack is implemented with arrays , top acts as a index to array.
When stack is implemented using link list top acts as a pointer to lastest added node.

Stack With arrays :

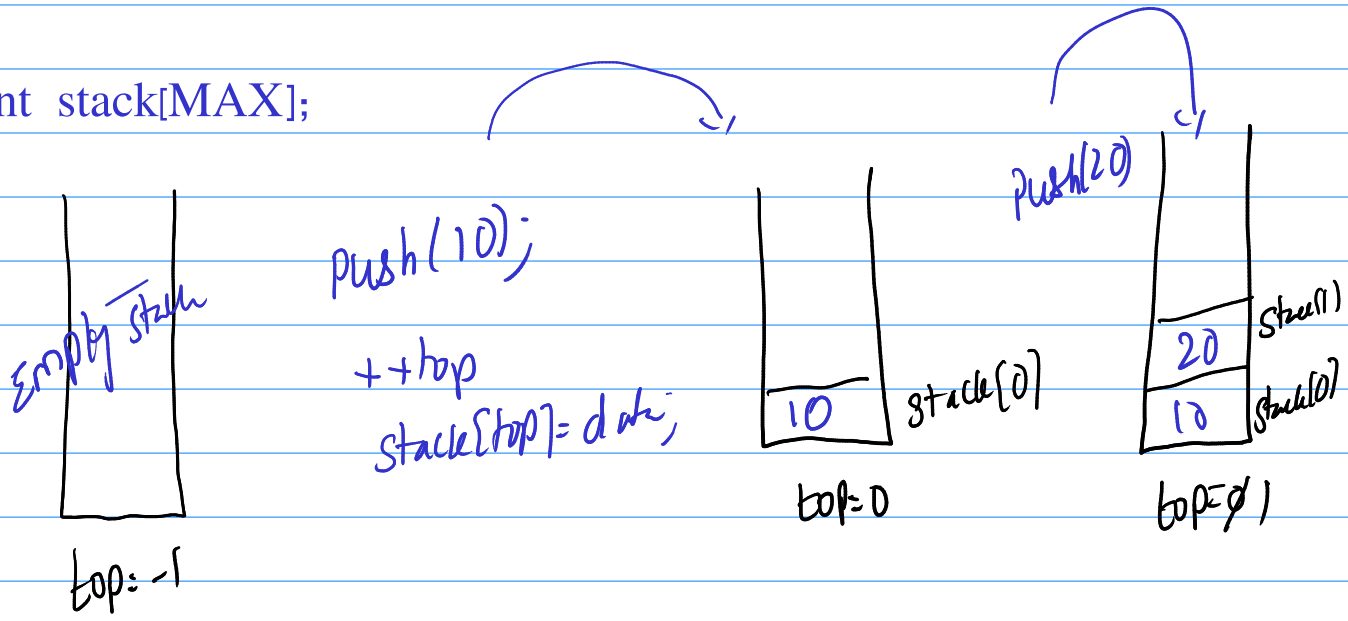
algorithm for push
1. check for stack full
2. if not full increment the top and insert the data at top inde

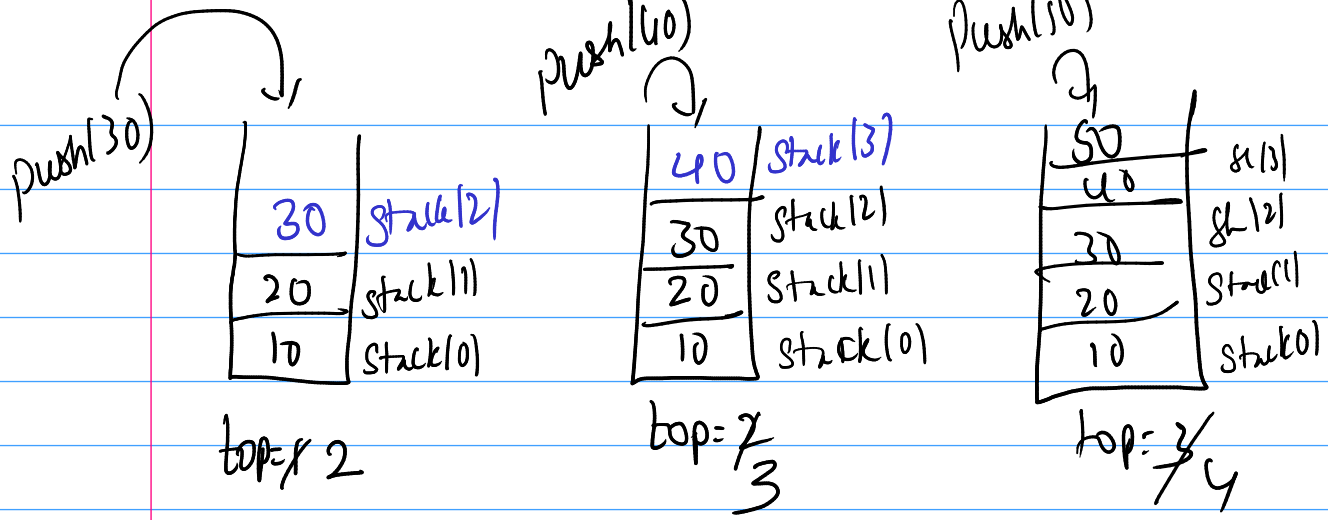
algorithm for pop

1. is empty
2. if not empty ,delete the data and decrement the top

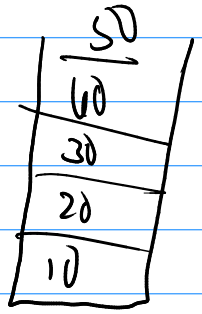
algorithm for peek
1. check for is empty
2. if not empty , return data pointed by top of the stack

int stack[MAX];



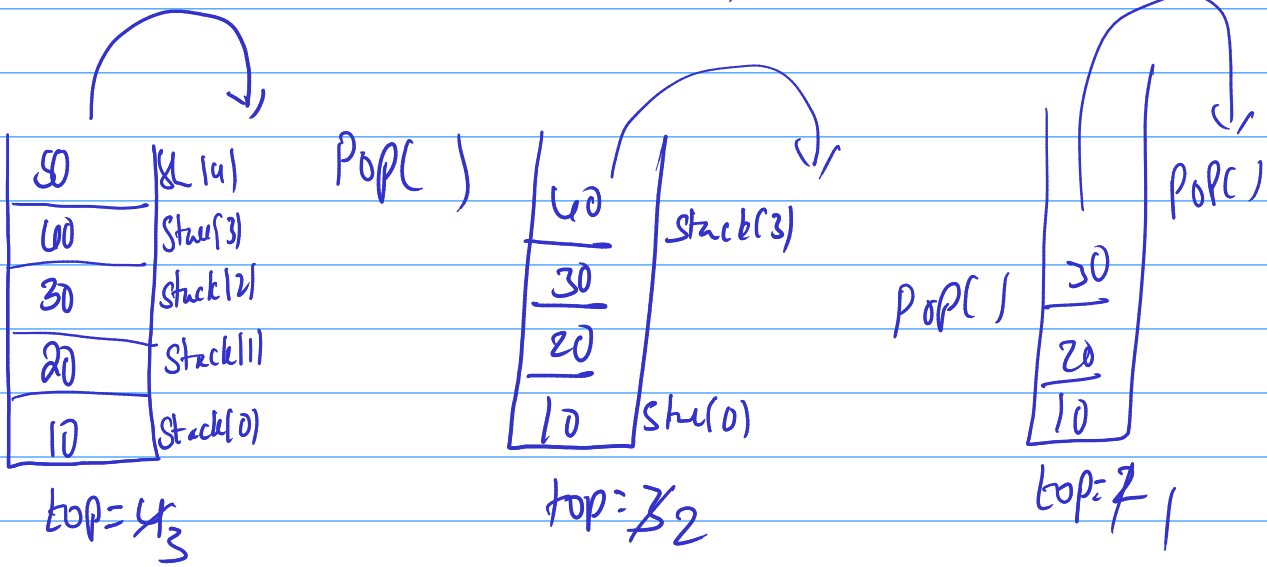



```
if(top==MAX-1)
    Stack full
else
    stack[++top]=data;
```



push(60)

=



Pop

return Stack[top--];

Peek()

return Stack[top];

Stack with Link List

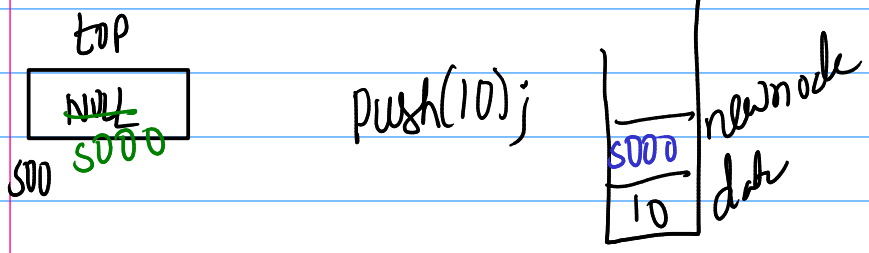
Push add at top addfirst
Pop delete at top or delete first

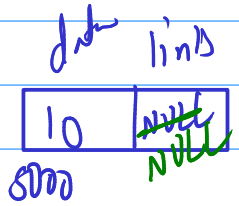
Algorithm for Push

1.create a node
2. intialise the node with given data.
3. assign top to newnode link
   newnode to top

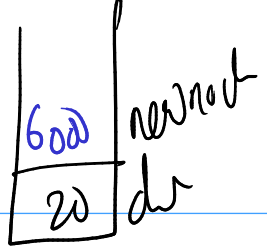
Algorithm for Pop

1. check stack empty
2. if not empty , assign top to temp, then move top to next node, free temp



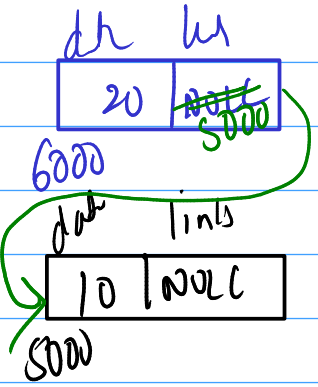


push(20);

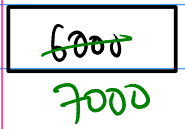


top

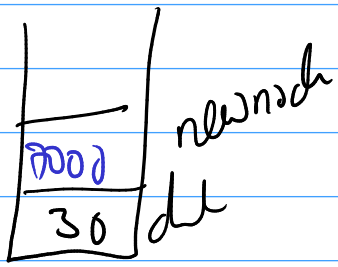


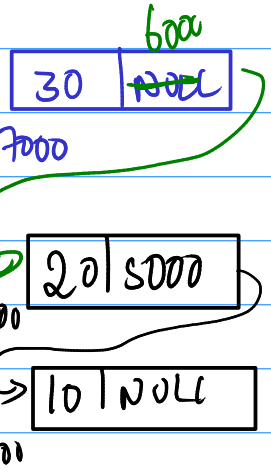


top

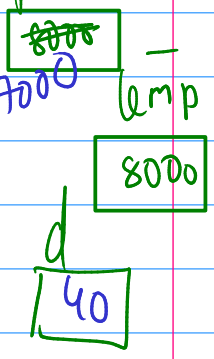


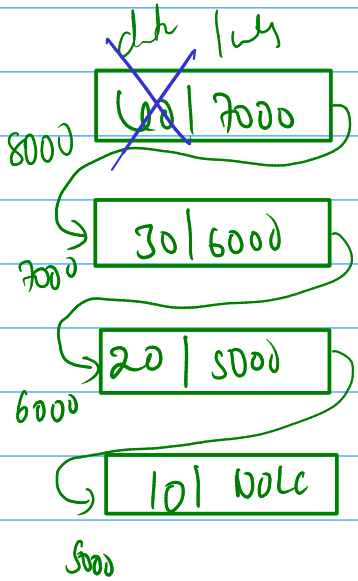
push(30);





top






```
temp = top;
top = top->link;
d = temp->data;
free(temp);
return d;
```